Anirudh Ramesh
PES1UG24CS929
OS project - Orange
16/04/2026

Phase 1 screenshots

1A)

```
➜  PES1UG24CS929-pes-vcs git:(main) ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
➜  PES1UG24CS929-pes-vcs git:(main)
```

1B)

```
➜  PES1UG24CS929-pes-vcs git:(main) find .pes/objects -type f | sort
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
➜  PES1UG24CS929-pes-vcs git:(main)
```

Phase 2 screenshots

2A)

```
→  PES1UG24CS929-pes-vcs git:(main) make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
→  PES1UG24CS929-pes-vcs git:(main) ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B)

```
→  PES1UG24CS929-pes-vcs git:(main) ✗ xxd .pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
00000000: 7472 6565 2034 3900 3130 3036 3634 2068  tree 49.100664 h
00000010: 656c 6c6f 2e74 7874 0066 2246 63d2 3e6f  ello.txt.f"Fc.>o
00000020: 4d9d e9e2 c7e6 d876 4305 a92a 3830 a1a5  M......vC..*80..
00000030: 2d3d 5f4a a800 7b5c 39                   -=_J..{\9
```

Phase 3 screenshots

3a)

```
→  PES1UG24CS929-pes-vcs git:(main) ./pes init
Initialized empty PES repository in .pes/
```

```
● → PES1UG24CS929-pes-vcs git:(main) ✗ echo "world" > file2.txt
● → PES1UG24CS929-pes-vcs git:(main) ✗ ./pes add file1.txt file2.txt
● → PES1UG24CS929-pes-vcs git:(main) ✗ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  test_objects
  untracked:  object.c
  untracked:  index.c
  untracked:  README.md
  untracked:  test_tree
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  pes.h
  untracked:  tree.h
  untracked:  index.h
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  tree.c
  untracked:  test_sequence.sh
```

Screenshot 3b

```
→ PES1UG24CS929-pes-vcs git:(main) ✗ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776322383 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776322414 6 file2.txt
→ PES1UG24CS929-pes-vcs git:(main) ✗
```

Phase 4 screenshots

4a

```
➜  PES1UG24CS929-pes-vcs git:(main) ✗ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 6a9238eb5d7e... Initial commit
Committed: 983facd1f792... Add world
Committed: da74b46bedfe... Add farewell
commit da74b46bedfe7c714f5374954613dd25a3904a89e09df4bf5d2fd0cc64210e90
Author: PES User <pes@localhost>
Date:   1776324645

    Add farewell

commit 983facd1f7922dce209564c34168d75edc9dde1b72ac0eddfe2cced9046c6376
Author: PES User <pes@localhost>
Date:   1776324645

    Add world

commit 6a9238eb5d7e128edb361072d500aa0d026f29c9303d94e885a054de75911bb3
Author: PES User <pes@localhost>
Date:   1776324645

    Initial commit

No commits yet.
```

4b

```
→  PES1UG24CS929-pes-vcs git:(main) ✗ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/25/3b4cd13a3443f842b0f2558354a19048ea55107a255d77238c176fa7c5a576
.pes/objects/4b/119fad4965d82c493db0d138645f4a2af15e89e041cbc91bd7e1ce973b1027
.pes/objects/50/da219c4c66a82940f7a8555908504a9bb1ede3773efe8669ad72f1d54966b3
.pes/objects/5c/8cc9820f5b7ba63d7a88240baf973a03a2cd323537f7f17184a7f519dea4e7
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6a/9238eb5d7e128edb361072d500aa0d026f29c9303d94e885a054de75911bb3
.pes/objects/75/157a044bc2cba79e457703e090fd58866d72ee3529c376909ae7f1c297e25e
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
.pes/objects/8f/fb561e2f4aa7f6df4e76dafeccd7b25eeb5b675876354e35d276192d26c446
.pes/objects/98/3facd1f7922dce209564c34168d75edc9dde1b72ac0eddfe2cced9046c6376
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/objects/cc/bf36e56826fe82981f163dcc989bc4ee82cb304ab49fafe958b3337d1d004e
.pes/objects/da/74b46bedfe7c714f5374954613dd25a3904a89e09df4bf5d2fd0cc64210e90
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f4/3354c88595ab525d304d08680ccc542a81d7159d5906a7da6c3ce720f65f30
.pes/refs/heads/main
```

4c

```
.pes/refs/heads/main
→  PES1UG24CS929-pes-vcs git:(main) ✗ cat .pes/refs/heads/main
da74b46bedfe7c714f5374954613dd25a3904a89e09df4bf5d2fd0cc64210e90
→  PES1UG24CS929-pes-vcs git:(main) ✗ cat .pes/HEAD
ref: refs/heads/main
→  PES1UG24CS929-pes-vcs git:(main) ✗
```

Phase 5 & 6: Analysis Questions

Branching and Checkout

Q5.1: Implementing pes checkout <branch>
To implement checkout, the following files must change in .pes/:

1. Read the target branch file from .pes/refs/heads/<branch> to get the commit hash
2. Update .pes/HEAD to point to "ref: refs/heads/<branch>"
3. Read the commit object to find its tree hash
4. Recursively reconstruct all files from the tree and blob objects into the working directory

The working directory must be completely rewritten to match the target tree. This operation is complex because:

- All files must be validated against the target tree before any changes occur (atomic operation)
- File mode bits must be preserved correctly
- If a file exists in the current tree but not in the target tree, it must be deleted
- If a file exists in the target tree but not the current tree, it must be created

- Performing this as a true atomic operation is difficult to guarantee without transaction support

Q5.2: Detecting Dirty Working Directory Conflicts

To detect conflicts without checking the working directory, compare the index against the target branch's tree:

1. Read the target commit's tree hash
2. Recursively traverse the target tree, building a hash map of (path -> hash, mode)
3. For each entry in the index, check if it exists in the target tree with a different hash or mode
4. For each entry in the target tree, check if it exists in the index with a different hash or mode
5. If any mismatches are found, the working directory is dirty relative to checkout and the operation must abort

This works because the index represents what files are staged and their hashes. Any difference between the current index and the target tree indicates uncommitted changes.

Q5.3: Detached HEAD State

When HEAD contains a commit hash directly (not a branch reference), commits made from this state create a new commit object with the detached HEAD as parent. However, no branch file is updated to point to this new commit. After making commits, if the user switches branches or runs gc, these new commits become unreachable.

To recover commits made in detached HEAD state, the user can search the reflog (a log of all HEAD movements) or use tools like git fsck to find dangling commits (commits with no branch pointing to them). The commits still exist in the object store and can be recovered by creating a new branch pointing to one of these orphaned commits.

Garbage Collection and Space Reclamation

Q6.1: Algorithm for Finding and Deleting Unreachable Objects

Algorithm:

1. Start with all branch refs in .pes/refs/heads/ and read their commit hashes
2. Use breadth-first search (or depth-first search) to traverse from each branch commit:
    - Visit the commit object and read its tree hash
    - Visit the tree object and recursively visit all referenced tree and blob objects
    - Mark each visited object hash as reachable
3. Scan the entire object store and delete any object whose hash was not marked as reachable

Efficient data structure: Use a hash set to track reachable hashes with O(1) lookup time.

For a repository with 100,000 commits and 50 branches:

- Traverse 50 commit objects (one per branch)
- Each commit points to one tree, and trees reference blobs and other trees
- In the worst case with 100,000 commits, estimate approximately 150,000 to 300,000 objects (commits + trees + blobs)
- Must visit each reachable object once, so approximately 150,000 to 300,000 object visits

Q6.2: Race Condition in Concurrent GC and Commit

Dangerous scenario:

1. Concurrent commit creates a new commit object with hash C3 and is about to update HEAD to point to C3
2. Simultaneously, garbage collection traverses reachable objects and has not yet visited C3 (because HEAD still points to C2)
3. GC completes and deletes C3 as "unreachable" (it was not yet reachable at the time of traversal)
4. Commit tries to update HEAD to C3, but C3 no longer exists in the object store, causing repository corruption